

Java Assignment 8

Exception Handling

Total Marks = 20 Marks

Question 1: Banking Transaction System [10 Marks]

Design a **Bank Fund Transfer** module with the following classes: Account, Bank, AuditLog and BankApp (main class).

Requirements

1. Create the following **user-defined exceptions**:
 - InvalidAccountException (checked) – thrown when an account number does not exist.
 - InsufficientBalanceException (checked) – must store and return the **shortfall amount** through getShortfall().
 - DailyLimitExceededException (checked) – thrown when total withdrawals in a day exceed Rs. 20,000.
 - InvalidAmountException (**unchecked**) – thrown when a deposit or withdrawal amount is zero or negative.
 - TransactionFailedException (checked) – a wrapper exception that carries the original exception as its **cause**.
2. Account.withdraw() must declare the checked exceptions with throws and raise them using throw.
3. AuditLog must implement AutoCloseable and print messages when opened and closed. Use it in Bank.transfer() with **try-with-resources**.
4. Bank.transfer(from, to, amount) must use a **multi-catch** block for InvalidAccountException and DailyLimitExceededException, and a separate catch for InsufficientBalanceException. Every catch must **wrap and rethrow** a TransactionFailedException (exception chaining).
5. Add a finally block in transfer() that prints that the attempt has finished, whether it succeeded or failed.
6. In main(), use a loop with an **outer try-catch** and test 5 cases: successful transfer, daily limit exceeded, invalid account, insufficient balance, negative amount. For each failure print the message and the **root cause** class name.

Question 2: Course Registration System [10 Marks]

Design a **University Course Registration** module with classes Student, Course, Registrar and CourseApp (main class).

Requirements

7. Create the following **user-defined exceptions**:
 - InvalidStudentIdException (**unchecked**) – thrown from the Student constructor when the ID does not match the format S followed by 3 digits (e.g. S101).
 - CourseFullException (checked) – thrown when the course has no seats.

- PrerequisiteNotMetException (checked) – must store the **missing prerequisite** course code and provide getMissing().
 - CreditLimitExceededException (checked) – thrown when the student's total credits would exceed 8.
 - RegistrationException (checked) – wrapper exception that holds the original cause.
8. Registrar.enroll(student, courseCode) must declare the three checked exceptions using throws. If the course code is unknown, a NullPointerException must be raised using Objects.requireNonNull(..., message).
 9. Registrar.register(student, String... courseCodes) must enroll the student in several courses using a **nested try-catch**: the inner try handles CourseFullException (print "Skipped" and continue with the next course); the outer try handles the remaining exceptions.
 10. The outer catch must use **multi-catch** for PrerequisiteNotMetException | CreditLimitExceededException and a separate catch for NullPointerException. Both must throw a RegistrationException with the original exception as cause (**chaining**).
 11. A finally block must always print how many courses the student was enrolled in and the current credits.
 12. In main(), create a student inside a try block, call register(), and handle RegistrationException and InvalidStudentIdException separately. Test: a successful registration, a full course, a missing prerequisite, an unknown course code and an invalid student ID.